Estados de respuesta HTTP: interpretar lo que devuelve una API — versión ampliada

Empieza por la versión corta para localizar los estados principales. Los casos de API de esta guía amplían el repertorio; no son todos códigos que haya que memorizar ni funciones de Hello Server.

El estado HTTP resume el resultado de procesar una petición. No sustituye al cuerpo de respuesta, pero da una pista rápida al cliente: éxito, redirección, error atribuible a la petición o fallo del servidor.

En una API REST bien diseñada, el estado, las cabeceras y el cuerpo deben contar la misma historia. No basta con devolver siempre `200 OK` y meter el error en un JSON: el cliente, las pruebas, los navegadores, los proxies y las herramientas de observabilidad interpretan primero el código HTTP.

Lectura rápida

- `1xx` **— informativa:** el servidor continúa procesando.
- `2xx` **— éxito:** la petición se ha aceptado o completado. Ejemplos: `200 OK`, `201 Created`, `204 No Content`.
- `3xx` **— redirección o caché:** hay que usar otra representación o una versión cacheada. Ejemplos: `301 Moved Permanently`, `304 Not Modified`.
- `4xx` **— problema en la petición:** no se puede atender tal como llega. Ejemplos: `400 Bad Request`, `401 Unauthorized`, `404 Not Found`.
- `5xx` **— problema del servidor:** el servidor o una dependencia han fallado. Ejemplos: `500 Internal Server Error`, `503 Service Unavailable`.

Regla práctica: **si el cliente puede corregir la petición, suele ser 4xx; si el cliente no puede arreglarlo y debe esperar o avisar, suele ser 5xx**.

Códigos frecuentes en aplicaciones REST

Éxito y redirección

- `200 OK`**:** operación correcta con cuerpo de respuesta. Ejemplo: `GET /api/tasks/7` devuelve la tarea.
- `201 Created`**:** se ha creado un recurso nuevo. Ejemplo: `POST /api/tasks` crea una tarea y devuelve `Location: /api/tasks/7`.
- `202 Accepted`**:** la petición se acepta, pero se procesará después. Ejemplo: importar un CSV grande o lanzar una tarea en segundo plano.
- `204 No Content`**:** operación correcta sin cuerpo. Ejemplo: `DELETE /api/tasks/7` elimina la tarea.
- `206 Partial Content`**:** se entrega un rango de bytes de una representación. Ejemplo: descarga parcial de un archivo con `Range`.
- `301 Moved Permanently`**:** el recurso tiene una URI nueva permanente.
- `302 Found`**:** redirección temporal, habitual en flujos web tradicionales.
- `304 Not Modified`**:** el cliente puede reutilizar su copia cacheada.

Errores del cliente y del servidor

- `400 Bad Request`**:** petición sintácticamente incorrecta o incoherente. Ejemplo: JSON mal formado.
- `401 Unauthorized`**:** falta autenticación válida. Ejemplo: no se envía token o el token es inválido/caducado.
- `403 Forbidden`**:** el usuario está autenticado, pero no tiene permiso.
- `404 Not Found`**:** no se ofrece una representación para ese recurso. Ejemplo: `GET /api/tasks/999` cuando no existe.
- `405 Method Not Allowed`**:** la URI existe, pero ese método no está permitido.
- `409 Conflict`**:** la petición choca con el estado actual. Ejemplo: email ya existente o actualización de una versión antigua.
- `410 Gone`**:** el recurso existió, pero ya no está disponible.

- `412 Precondition Failed`: falla una condición enviada por el cliente, como un `If-Match` con un `ETag` antiguo.
- `415 Unsupported Media Type`: el servidor no acepta el formato del cuerpo.
- `422 Unprocessable Content`: el formato se entiende, pero las reglas de negocio no se cumplen.
- `429 Too Many Requests`: el cliente ha superado un límite de peticiones.
- `500 Internal Server Error`: fallo inesperado no controlado.
- `502 Bad Gateway`: un intermediario recibe una respuesta inválida de otro servidor.
- `503 Service Unavailable`: servicio no disponible temporalmente.

No hay que memorizar todos los códigos. Lo importante es aprender a elegir el código que mejor describe el resultado y a justificarlo.

Estados de éxito: no todo es `200 OK`

`200 OK`

Úsalo cuando la operación ha terminado correctamente y devuelves una representación en el cuerpo.

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":7,"title":"Repasar HTTP","done":false}
```

Es habitual en `GET`, búsquedas y operaciones que devuelven datos.

`201 Created`

Úsalo cuando la petición crea un recurso nuevo. Si puedes identificar la URI del recurso creado, envía también la cabecera `Location`.

```
HTTP/1.1 201 Created
Location: /api/tasks/7
Content-Type: application/json

{"id":7,"title":"Repasar HTTP","done":false}
```

En Spring Boot se suele expresar así:

```
URI location = URI.create("/api/tasks/" + saved.id());
return ResponseEntity.created(location).body(saved);
```

Devolver `200 OK` después de crear algo puede funcionar técnicamente, pero pierde información. `201 Created` dice al cliente que hay un recurso nuevo.

`202 Accepted`

Úsalo cuando aceptas la petición, pero el resultado no está listo todavía. Es útil para trabajos asíncronos: importar muchos datos, generar un informe o procesar una cola.

```
HTTP/1.1 202 Accepted
Location: /api/jobs/abc123
Content-Type: application/json

{"jobId":"abc123","status":"pending"}
```

No significa «todo ha salido bien»; significa «he aceptado empezar o encolar el trabajo».

`204 No Content`

Úsalo cuando la operación es correcta pero no hay cuerpo que devolver. Es común en `DELETE`, en algunas actualizaciones y en acciones que no necesitan contenido de salida.

```
return ResponseEntity.noContent().build();
```

Con `204` no debe enviarse cuerpo. Si necesitas devolver información, usa `200` con JSON.

Redirecciones y caché: `3xx`

En una API JSON de clase no suelen ser los estados protagonistas, pero conviene reconocerlos.

`301 Moved Permanently` indica que el recurso se ha movido de forma estable. `302 Found` se ve mucho en aplicaciones web tradicionales, por ejemplo al llevar al usuario a otra página después de un formulario.

`304 Not Modified` no es un error. Aparece cuando el cliente pregunta de forma condicional si una representación ha cambiado. Si no ha cambiado, el servidor puede responder `304` sin reenviar el cuerpo.

```
GET /api/products/42 HTTP/1.1
If-None-Match: "v3"

HTTP/1.1 304 Not Modified
ETag: "v3"
```

Errores del cliente: `4xx`

Un `4xx` no significa necesariamente que el usuario sea culpable. Significa que la petición que ha llegado no puede aceptarse tal como está: falta autenticación, la ruta no existe, el formato no sirve o las reglas de validación fallan.

`400 Bad Request`

Úsalo para peticiones que no se pueden interpretar correctamente: JSON roto, parámetros imposibles o combinaciones incoherentes.

```
POST /api/tasks HTTP/1.1
Content-Type: application/json

{"title":
```

El servidor no puede procesar ese cuerpo porque ni siquiera es JSON completo.

`401 Unauthorized` y `403 Forbidden`

Los nombres confunden un poco:

- `401 Unauthorized`: no hay autenticación válida. El cliente debe autenticarse o renovar credenciales.
- `403 Forbidden`: el servidor comprende la petición, pero rehúsa autorizarla. Puede ocurrir incluso si se conoce la identidad; el ejemplo siguiente es un usuario autenticado sin el permiso requerido.
- Si no envío token a `/api/admin/users`, el estado razonable es `401 Unauthorized`.
- Si envío un token de alumno a una ruta de administración, el estado razonable es `403 Forbidden`.

En el ejemplo de clase, distingue «faltan credenciales válidas» (`401`) de «hay identidad, pero no permiso» (`403`); no conviertas esos dos casos habituales en una regla absoluta de implementación para todos los sistemas.

`404 Not Found`

El servidor no proporciona una representación para esa URI. Puede ser porque el recurso no existe o porque la aplicación decide no revelar su existencia.

```
return repository.findById(id)
        .map(ResponseEntity::ok)
        .orElseGet(() -> ResponseEntity.notFound().build());
```

En un `GET /api/tasks/999`, `404` suele ser más claro que devolver `200` con `null`.

405 Method Not Allowed

La ruta puede existir, pero no para ese método. Por ejemplo, una API puede permitir `GET /api/tasks` y `POST /api/tasks`, pero no `DELETE /api/tasks`.

```
HTTP/1.1 405 Method Not Allowed
Allow: GET, POST
```

Si el servidor incluye `Allow`, informa de los métodos permitidos para ese recurso.

409 Conflict

Úsalo cuando la petición es comprensible, pero entra en conflicto con el estado actual del sistema.

Casos frecuentes:

- Crear un usuario con un email ya registrado.
- Confirmar una reserva que acaba de ocupar otra persona.
- Actualizar un recurso con una versión antigua.

`409` es mejor que `400` cuando el problema no es el formato, sino el choque con el estado actual.

415 Unsupported Media Type

Aparece cuando el cliente envía un cuerpo en un formato que el endpoint no acepta.

```
POST /api/tasks HTTP/1.1
Content-Type: text/plain

comprar pan
```

Si el endpoint espera JSON, la aplicación puede responder `415`. No es lo mismo que `400`: aquí el problema principal es el `Content-Type`.

422 Unprocessable Content

El servidor entiende el formato, pero el contenido no cumple las reglas. Es útil para errores de validación de dominio o validaciones de DTO.

```
{
  "title": "",
  "dueDate": "2020-01-01"
}
```

Si el JSON está bien formado pero `title` no puede estar vacío, puedes devolver `400` o `422` según el contrato de la API. Lo importante es ser coherente. En muchas aplicaciones Spring Boot sencillas se usa `400` para validación con `@Valid`; en APIs más expresivas se reserva `422` para reglas de negocio.

429 Too Many Requests

Indica que el cliente ha superado un límite. Suele acompañarse de cabeceras como `Retry-After` o información de rate limiting.

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
```

Es habitual en login, APIs públicas o integraciones donde hay que proteger el servicio.

Errores del servidor: `5xx`

Un `5xx` comunica que el servidor no ha podido completar una petición que, en principio, podía ser válida. No debe filtrar trazas, secretos, nombres internos de tablas ni detalles de infraestructura.

`500 Internal Server Error`

Fallo inesperado. En desarrollo conviene mirar logs y tests; en producción debe devolverse un cuerpo de error seguro.

```
{
  "status": 500,
  "error": "Internal Server Error",
  "message": "Unexpected server error",
  "path": "/api/tasks"
}
```

`502 Bad Gateway` y `503 Service Unavailable`

Estos estados aparecen cuando hay proxies, gateways o dependencias externas.

- `502`: un intermediario recibió una respuesta inválida de otro servidor.
- `503`: el servicio no está disponible temporalmente, por mantenimiento, saturación o caída de una dependencia.

En clase pueden verse menos que `400`, `404` o `500`, pero son importantes en despliegues reales.

Relación entre método y estado

El método orienta el estado esperado, pero no lo determina por completo.

- **`GET /api/tasks`:** éxito habitual `200 OK`; errores frecuentes `401`, `403`, `500`.
- **`GET /api/tasks/7`:** éxito habitual `200 OK`; errores frecuentes `401`, `403`, `404`, `500`.
- **`POST /api/tasks`:** éxito habitual `201 Created`; errores frecuentes `400`, `401`, `403`, `409`, `415`, `422`, `500`.
- **`PUT /api/tasks/7`:** éxito habitual `200 OK` o `204 No Content`; errores frecuentes `400`, `401`, `403`, `404`, `409`, `415`, `422`, `500`.
- **`PATCH /api/tasks/7`:** éxito habitual `200 OK` o `204 No Content`; errores frecuentes `400`, `401`, `403`, `404`, `409`, `415`, `422`, `500`.
- **`DELETE /api/tasks/7`:** éxito habitual `204 No Content`; errores frecuentes `401`, `403`, `404`, `409`, `500`.

No conviertas esta tabla en una receta mecánica. El estado correcto depende del contrato de la API y de lo que haya ocurrido realmente.

Cuerpo de error: qué debe explicar y qué no

Un estado como `400` o `404` dice la categoría del problema. El cuerpo puede dar detalles útiles para corregirlo.

Ejemplo razonable:

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "title must not be blank",
  "path": "/api/tasks"
}
```

Ejemplo peligroso:

```
{
  "exception": "java.sql.SQLSyntaxErrorException",
  "trace": "...",
  "sql": "select * from users where ..."
}
```

El cliente necesita saber que ha fallado y cómo corregirlo. No necesita trazas, secretos, rutas internas ni detalles de base de datos.

`206 Partial Content` no es paginación

`206 Partial Content` describe rangos de una representación: por ejemplo, bytes de un archivo solicitados con una cabecera `Range`.

```
GET /videos/demo.mp4 HTTP/1.1
Range: bytes=0-999

HTTP/1.1 206 Partial Content
Content-Range: bytes 0-999/5000000
```

Eso no equivale a paginar una colección REST:

```
GET /api/tasks?page=0&size=20 HTTP/1.1

HTTP/1.1 200 OK
Content-Type: application/json
```

Una página de resultados normalmente sigue siendo una representación completa de esa consulta, por eso se devuelve `200 OK`.

Exploración local

Con Hello Server arrancado, observa respuestas reales:

```
http --verbose GET http://localhost:8080/api/hello
http --verbose GET http://localhost:8080/no-existe
http --verbose POST http://localhost:8080/api/hello
```

Compara:

1. Código de estado.
2. Cabeceras relevantes (`Content-Type`, `Allow`, `Location`, si aparecen).
3. Cuerpo de respuesta.
4. Qué parte del resultado procede de HTTP y qué parte la decide la aplicación.

Comprobación

Para cada caso, elige un estado y justifícalo:

1. Se crea una tarea nueva y el servidor conoce su URI.
2. Se borra correctamente una tarea y no hay nada que devolver.
3. Se pide una tarea que no existe.
4. Se envía JSON mal formado.
5. Se envía JSON válido, pero el título obligatorio está vacío.
6. Un usuario autenticado intenta borrar un recurso que no es suyo.
7. Se intenta registrar un email que ya existe.
8. La base de datos está caída temporalmente.

La respuesta buena no es solo el número: también hay que explicar qué cuerpo y qué cabeceras ayudarían al cliente.